LiDAR Obstacle Avoidance

LiDAR Obstacle Avoidance

1. Course Content
2. Running the Cases
3. Core Source Code Analysis

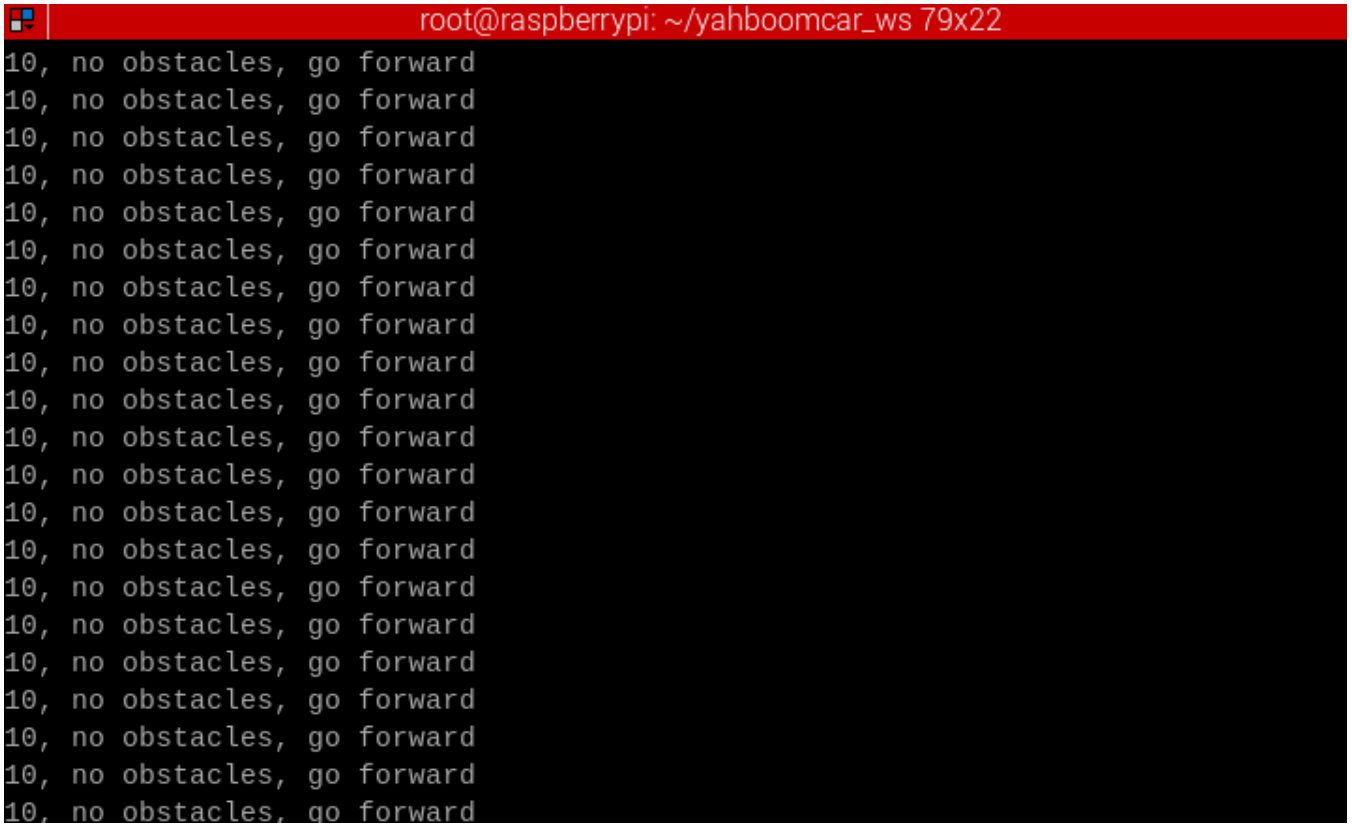
1. Course Content

This section explains how the program combines chassis control and fused LiDAR data to detect obstacles in real time while the car is moving forward, and control the car's turning based on the obstacle's position.

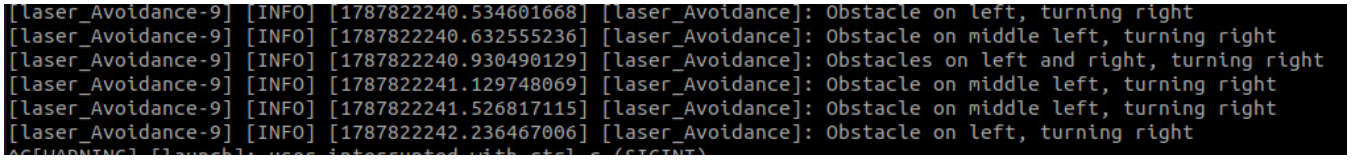
2. Running the Cases

```
ros2 launch yahboomcar_bringup lidar_demo.launch.py
```

After startup, as shown below, the detected surrounding obstacle status is printed, and the car's motion is controlled



If there is no obstacle, the car moves straight forward; if there is an obstacle on the left, the car rotates to the right to avoid the obstacle, then continues moving forward; if there is an obstacle on the right, the car rotates 90 degrees to the left to avoid the obstacle, then continues moving forward.



- To end the case, press Ctrl+C in the terminal, then close the window

3. Core Source Code Analysis

- Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/lidar/laser_Avoidance.py
```

- Automatically detects the LiDAR `frame_id` and establishes the `base_link ← laser_frame` coordinate transform chain via TF
- Resets the front, left, and right obstacle counters `front_warning / Left_warning / Right_warning` on each callback
- Iterates through all laser points, filters out invalid values, and transforms each point from the LiDAR coordinate frame to the `base_link` coordinate frame
- Based on the obstacle's angle under `base_link`, classifies and counts the number of obstacle points in front (`abs(angle) < LaserAngle`), on the left (`angle > 0`), and on the right (`angle < 0`)
- Based on the comparison of the three-side counters with the threshold `conut`, determines the current obstacle state and sets the corresponding velocity command (forward, turn left, turn right, etc.)
- Outputs logs only when the state changes, and finally publishes the `Twist` velocity command to `/cmd_vel`

```
    def registerScan(self, scan_data):
        if not isinstance(scan_data, LaserScan): return

        # Auto-detect laser frame_id
        if not self.laser_frame:
            self.laser_frame = scan_data.header.frame_id
            self.get_logger().info(f'Detected laser frame: {self.laser_frame}')

        # If TF is not ready, try to establish connection
        if not self.tf_ready:
            try:
                self.tf_buffer.lookup_transform(
                    'base_link',
                    scan_data.header.frame_id,
                    rclpy.time.Time(),
                    timeout=rclpy.duration.Duration(seconds=0.1)
                )
                self.tf_ready = True
                self.get_logger().info(f'TF ready: base_link <-
{scan_data.header.frame_id}')
            except TransformException:
                # TF not ready, skip this scan
                return
```

```
        # Reset counters
        self.Right_warning = 0
        self.Left_warning = 0
        self.front_warning = 0

        # Get transform from laser to base_link
        try:
            trans = self.tf_buffer.lookup_transform(
                'base_link',
                scan_data.header.frame_id,
                rclpy.time.Time(),
                timeout=rclpy.duration.Duration(seconds=0.1)
            )
        except TransformException:
            # Throttle TF error logs
            current_time = self.get_clock().now().nanoseconds / 1e9
            if (current_time - self.last_tf_log_time) >= self.tf_log_interval:
                self.get_logger().warn('TF not available, skipping scan')
                self.last_tf_log_time = current_time
            return

        # Iterate through all laser points
        ranges = np.array(scan_data.ranges)
        for i in range(len(ranges)):
            if ranges[i] == 0.0 or ranges[i] > scan_data.range_max:
                continue

            # Calculate laser point position in laser coordinate frame
            angle = scan_data.angle_min + scan_data.angle_increment * i
            point_in_laser = PointStamped()
            point_in_laser.header = scan_data.header
            point_in_laser.point.x = ranges[i] * math.cos(angle)
            point_in_laser.point.y = ranges[i] * math.sin(angle)
            point_in_laser.point.z = 0.0

            # Transform to base_link coordinate frame
            try:
                point_in_base = do_transform_point(point_in_laser, trans)
                x = point_in_base.point.x
                y = point_in_base.point.y

                # Determine obstacle position based on base_link coordinate frame
                dist = math.sqrt(x**2 + y**2)
                if dist <= self.ResponseDist * 1.5:
                    # Use atan2 to calculate angle relative to base_link x-axis
                    angle_to_base = math.atan2(y, x) * RAD2DEG

                    if abs(angle_to_base) < self.LaserAngle:
                        self.front_warning += 1
                    elif angle_to_base > 0:
                        self.Left_warning += 1
                    else:
                        self.Right_warning += 1
```

```
            except TransformException:
                continue
        twist = Twist()
        current_time = self.get_clock().now().nanoseconds / 1e9

        # Determine current obstacle state
        obstacle_state = 0
        if self.front_warning > self.conut and self.Left_warning > self.conut and
self.Right_warning > self.conut:
            obstacle_state = 1
            twist.linear.x = self.linear
            twist.angular.z = -self.angular
        elif self.front_warning > self.conut and self.Left_warning <= self.conut and
self.Right_warning > self.conut:
            obstacle_state = 2
            twist.linear.x = 0.0
            twist.angular.z = self.angular
        elif self.front_warning > self.conut and self.Left_warning > self.conut and
self.Right_warning <= self.conut:
            obstacle_state = 4
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
        elif self.front_warning > self.conut and self.Left_warning < self.conut and
self.Right_warning < self.conut:
            obstacle_state = 6
            twist.linear.x = 0.0
            twist.angular.z = self.angular
        elif self.front_warning < self.conut and self.Left_warning > self.conut and
self.Right_warning > self.conut:
            obstacle_state = 7
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
        elif self.front_warning < self.conut and self.Left_warning > self.conut and
self.Right_warning <= self.conut:
            obstacle_state = 8
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
        elif self.front_warning < self.conut and self.Left_warning <= self.conut and
self.Right_warning > self.conut:
            obstacle_state = 9
            twist.linear.x = 0.0
            twist.angular.z = self.angular
        elif self.front_warning <= self.conut and self.Left_warning <= self.conut and
self.Right_warning <= self.conut:
            obstacle_state = 10
            twist.linear.x = self.linear
            twist.angular.z = 0.0

        # Log only when state changes or after log_interval
        if obstacle_state != self.last_obstacle_state or (current_time - self.last_log_time)
>= self.log_interval:
            log_messages = {
                1: 'Obstacles on left and right, turning right',
```

```
                2: 'Obstacle on middle right, turning left',
                4: 'Obstacle on middle left, turning right',
                6: 'Obstacle in front, turning left',
                7: 'Obstacles on left and right, turning right',
                8: 'Obstacle on left, turning right',
                9: 'Obstacle on right, turning left',
                10: 'No obstacles, going forward'
            }
            if obstacle_state in log_messages:
                self.get_logger().info(log_messages[obstacle_state])
            self.last_log_time = current_time
            self.last_obstacle_state = obstacle_state

        self.pub_vel.publish(twist)
```